

Practical 1

Name: Tanvi Santosh Gundale

PRN: 202501100094

Division: SE1

Batch : S15

EDS Practical No :- 1

1.1.1 Calculate Momentum:

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula:

$$P = m \cdot v$$

Input Format:

A single floating-point number representing the mass of the object in kilograms.

A single floating-point number representing the velocity of the object in meters per second.

Output Format:

The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places)

Code:

```
mass = float(input())
velocity = float(input())
momentum = mass*velocity
print(f"{momentum:.2f}kgm/s")
```

Output:

The screenshot displays a test runner interface with the following components:

- Summary:**
 - Average time: **0.013 s** (13.25 ms)
 - Maximum time: **0.028 s** (28.00 ms)
 - 2 out of 2 shown test case(s) passed
 - 2 out of 2 hidden test case(s) passed
- Test case 1 (28 ms):**
 - Buttons: Debug, menu, table
 - Expected output: 5.0, 10.0, 50.00kgm/s
 - Actual output: 5.0, 10.0, 50.00kgm/s
- Test case 2 (7 ms):**
 - Buttons: Debug, menu, table
 - Expected output: 2.3, 4.8, 11.04kgm/s
 - Actual output: 2.3, 4.8, 11.04kgm/s

1.1.2 Conditional Calculations:

Write a Python program that accepts an integer n as input. Depending on the number of digits in n . Constraints: $1 \leq n \leq 999$ Input format :

- The input consists of a single integer n .

Output format:

- If n is a single-digit number, print its square.
- If n is a two-digit number, print its square root (rounded to two decimal places).

- If n is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid".

Code: `n =`

`int(input()) if`

`1<=n<=9: print(n*n)`

`elif 10<=n<=99:`

`print(f"{n**(1/2) :.2f}") elif 100<=n<=999:`

`print(f"{n**(1/3):.2f}")`

`else :`

`print("Invalid"`

`)`

Output:

The screenshot displays a test runner interface with the following details:

- Summary:** 4 out of 4 shown test case(s) passed, 3 out of 3 hidden test case(s) passed.
- Performance:** Average time 0.009 s (8.57 ms), Maximum time 0.019 s (19.00 ms).
- Test Case 1:** Expected output 9, Actual output 9. Time: 19 ms.
- Test Case 2:** Expected output 166, Actual output 166. Time: 11 ms.
- Test Case 3:** Expected output 24, Actual output 24. Time: 6 ms.
- Test Case 4:** Expected output 3456, Actual output 3456. Time: 8 ms.

1.1.3 Days Between two Dates:

Write a Python program to calculate number of days between two dates.

Input Format:

The first line contains the first date in the format : YYYY-MM-DD

The second line contains the second date in the format :YYYY-MM-DD

Output Format:

An integer representing the number of days between the given dates. Note: The first date should always be considered earlier than the second date

Code:

```
from datetime import date
date1 = input().strip()
date2 = input().strip()
d1 = date.fromisoformat(date1)
d2 = date.fromisoformat(date2)
diff = (d2-d1).days
print(diff)
```

Output:—

The screenshot displays a test runner interface. At the top, it shows performance metrics: Average time 0.012 s (12.00 ms) and Maximum time 0.019 s (19.00 ms). It also indicates that 2 out of 2 shown test case(s) passed and 2 out of 2 hidden test case(s) passed. Below this, two test cases are listed. Test case 1, with a duration of 19 ms, shows an expected output of '2014-07-02', '2014-11-02', and '123', which matches the actual output. Test case 2, with a duration of 13 ms, shows an expected output of '2014-07-02', '2014-07-11', and '9', which also matches the actual output. Each test case entry includes a 'Debug' button and icons for menu and table views.

Performance Metrics		Test Results Summary	
Average time	0.012 s 12.00 ms	Maximum time	0.019 s 19.00 ms
		2 out of 2 shown test case(s) passed	
		2 out of 2 hidden test case(s) passed	

Test Case	Duration	Expected Output	Actual Output
Test case 1	19 ms	2014-07-02 2014-11-02 123	2014-07-02 2014-11-02 123
Test case 2	13 ms	2014-07-02 2014-07-11 9	2014-07-02 2014-07-11 9

1.1.4 Reverse a Number:

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

The input is a single integer.

Output Format:

The output prints a single integer, which is the reversed number

Code:

```
n = int(input()) rev = 0 while n>0:
    digit = n % 10    rev = rev*10 + digit    n = n // 10 print(rev)
```

Output:

Average time
0.006 s
6.40 ms

Maximum time
0.009 s
9.00 ms

✓ 2 out of 2 shown test case(s) passed
✓ 3 out of 3 hidden test case(s) passed

✓ Test case 1 6 ms Debug

Expected output

5367

7635

Actual output

5367

7635

✓ Test case 2 7 ms Debug

Expected output

0132

231

Actual output

0132

231

1.1.5 Multiplication Table:

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:

Code:

Output:

The screenshot shows a test runner interface with the following details:

- Performance Metrics:**
 - Average time: 0.005 s (5.50 ms)
 - Maximum time: 0.008 s (8.00 ms)
- Test Results Summary:**
 - 2 out of 2 shown test case(s) passed
 - 2 out of 2 hidden test case(s) passed
- Test Case 1:** 8 ms, Debug mode.
 - Expected output:** 8
 - Actual output:** 8
 - Comparison rows (all match):
 - 8 · x · 1 · = · 8
 - 8 · x · 2 · = · 16
 - 8 · x · 3 · = · 24
 - 8 · x · 4 · = · 32
 - 8 · x · 5 · = · 40
 - 8 · x · 6 · = · 48
 - 8 · x · 7 · = · 56
 - 8 · x · 8 · = · 64
 - 8 · x · 9 · = · 72
 - 8 · x · 10 · = · 80
- Test Case 2:** 4 ms

```
n = int(input()) for i in range(1,11): print(f"{n} x {i}  
= {n*i}")
```

1.2.1 Pass or Fail:

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.

```

Code: n = int(input()) marks = list(map(int ,
input().split())) if any
(m<40 for m in marks):
print("Fail")
else:
ave = sum(marks)/n    print("Aggregate
Percentage: %0.2f"%ave)          if ave>75:
print("Grade: Distinction")      elif 60 <= ave < 75:
print("Grade: First Division")  elif 50 <= ave < 60:    print("Grade:
Second Division")    elif 40 <= ave < 50:
print("Grade: Third Division")

```

Output:

Average time
0.007 s
7.10 ms

Maximum time
0.010 s
10.00 ms

5 out of 5 shown test case(s) passed

5 out of 5 hidden test case(s) passed

Test case 19 ms

Debug



Expected output

4

55 65 78 89

Aggregate Percentage: 71.75

Grade: First Division

Actual output

4

55 65 78 89

Aggregate Percentage: 71.75

Grade: First Division

Test case 27 ms

Debug



Expected output

5

56 78 97 86 93

Aggregate Percentage: 82.00

Grade: Distinction

Actual output

5

56 78 97 86 93

Aggregate Percentage: 82.00

Grade: Distinction

Test case 38 ms

Debug



Expected output

5

99 85 43 97 34

Fail

Actual output

5

99 85 43 97 34

Fail

1.2.2 Fibonacci Series:

Write a Python program that uses recursion to print the first terms of the Fibonacci series.

Input Format:

- A single integer representing the number of terms to generate.

Output Format:

- A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

Code:

```
def fibonacci(n):  
    if n==1:        return 0    elif  
    n==2:        return 1    else :  
        return fibonacci(n-1) + fibonacci(n-2)  
2) n = int(input())  
    for i in range(1, n + 1):  
        print(fibonacci(i), end=" ")
```

Output :

Average time
0.018 s
18.50 ms

Maximum time
0.045 s
45.00 ms

✓ 2 out of 2 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 10 ms Debug

Expected output
5
0 1 1 2 3

Actual output
5
0 1 1 2 3

✓ Test case 2 5 ms Debug

Expected output
15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

Actual output
15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

1.2.3 Pattern-1:

Write a Python program to print a pattern of asterisks in the form of a rightangled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Code:

```
n = int(input())
for i in range(1,n+1):
    for j in range(1,i+1):
        print("*",end=" ")
    print()
```

The screenshot displays a code execution environment with the following details:

- Performance Metrics:**
 - Average time: 0.004 s (4.33 ms)
 - Maximum time: 0.005 s (5.00 ms)
- Test Results:**
 - 2 out of 2 shown test case(s) passed
 - 4 out of 4 hidden test case(s) passed
- Test Case 1 (n=5):**
 - Expected output:

```
5
*
* *
* * *
* * * *
* * * * *
```
 - Actual output: (Matches expected output)
- Test Case 2 (n=4):**
 - Expected output:

```
4
*
* *
* * *
* * * *
```
 - Actual output: (Matches expected output)

1.2.4 Pattern - 2 :

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Code:

```
n = int(input()) for i in range(1,n+1): for j in range(1,i+1):  
    print(j,end=" ")    print()
```

Output:

Average time
0.007 s
7.00 ms

Maximum time
0.008 s
8.00 ms

✓ 2 out of 2 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 8 ms Debug

Expected output

Actual output

5	5
1	1
1 · 2	1 · 2
1 · 2 · 3	1 · 2 · 3
1 · 2 · 3 · 4	1 · 2 · 3 · 4
1 · 2 · 3 · 4 · 5	1 · 2 · 3 · 4 · 5

✓ Test case 2 6 ms Debug

Expected output

Actual output

8	8
1	1
1 · 2	1 · 2
1 · 2 · 3	1 · 2 · 3
1 · 2 · 3 · 4	1 · 2 · 3 · 4
1 · 2 · 3 · 4 · 5	1 · 2 · 3 · 4 · 5
1 · 2 · 3 · 4 · 5 · 6	1 · 2 · 3 · 4 · 5 · 6
1 · 2 · 3 · 4 · 5 · 6 · 7	1 · 2 · 3 · 4 · 5 · 6 · 7
1 · 2 · 3 · 4 · 5 · 6 · 7 · 8	1 · 2 · 3 · 4 · 5 · 6 · 7 · 8

